



ARQUITETURA DE SISTEMAS

AULA 5 - PENSAMENTO ORIENTADO A COMPONENTES



O QUE É UM COMPONENTE?

- ❑ Módulo vs Componente: Um módulo é um agrupamento lógico de código relacionado; já um componente é a manifestação física desse módulo, um pacote de código pronto para implantar (ex.: arquivos .jar no Java, .dll no .NET). Em resumo, componentes são unidades de implantação que contêm um ou mais módulos do sistema.
- ❑ Exemplos de Componentes: Linguagens fornecem mecanismos para empacotar módulos fisicamente como componentes: por exemplo, bibliotecas (libraries) compiladas e vinculadas em tempo de compilação, serviços independentes implantáveis, plugins, etc. Esses componentes permitem distribuir funcionalidades do sistema de forma modular e reutilizável.



ESCOPO DE UM COMPONENTE

- ❑ Biblioteca (Library): Componente mais simples, empacota código reutilizável que roda no mesmo processo da aplicação cliente, acessado via chamadas de função/método. Geralmente introduz dependência em tempo de compilação (ex.: arquivos JAR). Útil para organizar e reutilizar funcionalidades comuns.
- ❑ Subsistema/Camada: Componente que agrupa várias classes/módulos em um nível mais alto, podendo definir uma layer ou serviço interno do sistema. Normalmente faz parte de uma aplicação maior (monólito) e interage com outros componentes via chamadas internas, organizando o sistema em camadas lógicas (por ex.: apresentação, negócio, persistência).
- ❑ Serviço (Service): Componente autônomo executado em seu próprio processo ou servidor, comunicando-se via rede (protocolos como REST, filas de mensagem, TCP/IP). Representa uma unidade implantável isolada (ex.: um microservice) com seu próprio contexto, ideal para escalabilidade independente e isolamento de falhas.



PAPÉIS NA DEFINIÇÃO DE COMPONENTES

- ❑ Papel do Arquiteto: Define e é responsável pelos componentes do sistema em alto nível, colaborando com analistas de negócio, especialistas e desenvolvedores. O arquiteto decide a divisão top-down do sistema em componentes, considerando requisitos funcionais e características arquiteturais (requisitos não-funcionais). Ele evita se ater aos detalhes de classe internos de cada componente (exceto em casos de aplicar padrões de design), delegando essas decisões aos tech leads e desenvolvedores para não microgerenciar tudo.
- ❑ Papel do Desenvolvedor: Refina os componentes em estruturas de código (classes, funções) e implementa os detalhes internos. Desenvolvedores e líderes técnicos participam do design de componentes junto com o arquiteto, tratando o esboço inicial de componentes como um primeiro rascunho a ser iterativamente melhorado. Ou seja, à medida que o desenvolvimento avança, a equipe pode descobrir necessidade de ajustar a divisão de componentes, garantindo que o design evolua conforme surgem novos insights.



PARTICIONAMENTO ARQUITETURAL

- ❑ Divisão em Componentes: Decidir como particionar a arquitetura em componentes de alto nível é uma das primeiras tarefas do arquiteto. Não há uma única forma correta: toda decisão envolve trade-offs (a Primeira Lei da Arquitetura de Software). Assim, o arquiteto deve avaliar diferentes estilos de particionamento e seus impactos.
- ❑ Estilos de Particionamento: Dois estilos comuns de organização global do sistema são a partição técnica em camadas e a partição por domínios de negócio. Por exemplo, um monólito tradicional em camadas separa a aplicação em camadas técnicas (UI, regras de negócio, acesso a dados), enquanto um monólito modular (inspirado em Domain-Driven Design) agrupa lógicas por domínio de negócio em componentes distintos. Cada estilo traz benefícios e desafios diferentes, e definir a partição correta é fundamental para a estrutura da arquitetura.



PARTICIONAMENTO POR DOMÍNIO (DOMAIN PARTITIONING)

- ❑ Foco no Domínio: A arquitetura é dividida em componentes autônomos alinhados a workflows ou contextos de negócio específicos (ex.: CatalogCheckout, Analytics, Inventory). Essa estratégia aproxima o software do modelo do mundo real, facilitando a evolução independente de cada domínio.
- ❑ Vantagens: Maior coesão de negócio: cada componente implementa um conjunto de casos de uso do domínio, reduzindo dependências entre domínios. O design fica mais próximo de um monólito modular ou arquitetura de microservices, com fluxo de mensagens alinhado às funcionalidades reais do problema. Além disso, fica mais fácil migrar gradualmente partes da aplicação para um sistema distribuído (microserviços) caso necessário.
- ❑ Desafio: Possível duplicação de código ou lógica em múltiplos componentes quando funcionalidades semelhantes aparecem em domínios diferentes. Por exemplo, regras transversais (como validações comuns) podem precisar ser repetidas em componentes de domínios distintos se não houver um componente compartilhado, exigindo governança para evitar inconsistências.



PARTICIONAMENTO TÉCNICO EM CAMADAS

- ❑ Foco Técnico: A arquitetura é organizada por capacidades técnicas ou camadas lógicas, separando responsabilidades como apresentação, lógica de negócio, serviços e persistência. Todos os domínios de negócio atravessam essas camadas, centralizando funcionalidades técnicas comuns (ex.: uma camada única de banco de dados atende a diferentes módulos do sistema).
- ❑ Vantagens: Promove separação de preocupações técnicas: código de apresentação isolado do de negócio, que por sua vez é isolado do de dados. Facilita reutilizar componentes técnicos comuns (ex.: um módulo comum para regras genéricas) e segue um modelo familiar de arquitetura em camadas. Customizações específicas ficam claramente segregadas em módulos locais, enquanto partes comuns permanecem centralizadas, tornando a manutenção de funcionalidades transversais mais simples.



PARTICIONAMENTO TÉCNICO EM CAMADAS

- ❑ Desvantagens: Acoplamento global elevado: se partes comuns e partes locais estão fortemente ligadas, mudanças em um componente comum ou em um componente local podem impactar todos os outros. Muitas vezes há duplicação de conceitos de domínio entre a lógica comum e as customizações locais. Além disso, costuma-se compartilhar um único banco de dados para toda a aplicação, o que pode dificultar separar os componentes no futuro (forte dependência de dados compartilhados). Isso torna a migração para uma arquitetura distribuída mais complexa caso venha a ser necessária.



FLUXO DE IDENTIFICAÇÃO DE COMPONENTES

- ❑ Identificar componentes iniciais: Antes de codificar, o arquiteto propõe um conjunto inicial de componentes que cubra as principais funcionalidades, de acordo com o estilo de partição escolhido (técnica ou domínio). Essa primeira divisão é hipotética e dificilmente perfeita, servindo como ponto de partida para o design.
- ❑ Alocar requisitos aos componentes: Mapeia-se cada requisito ou user story para um componente, verificando se aquele componente deve assumir aquela funcionalidade. Nessa fase, possivelmente surgem novos componentes, fusões ou divisões de componentes existentes para melhor acomodar as responsabilidades mapeadas.
- ❑ Analisar papéis e responsabilidades: Avalia-se se cada componente tem um conjunto coeso de responsabilidades condizentes com os atores (usuários/roles) e casos de uso levantados. Deve-se checar se a granularidade está adequada: nem muito ampla (um componente fazendo coisas demais), nem muito estreita. Os componentes devem refletir limites claros de contexto de responsabilidades no sistema.



FLUXO DE IDENTIFICAÇÃO DE COMPONENTES

- ❑ Considerar características arquiteturais: Verifica-se como requisitos não-funcionais (desempenho, escalabilidade, segurança, etc.) afetam a divisão de componentes. Componentes que enfrentam exigências arquiteturais muito distintas talvez precisem ser separados. Exemplo: se duas funcionalidades são logicamente semelhantes mas uma requer suportar milhares de usuários e outra apenas poucos, pode ser benéfico separá-las em componentes diferentes para otimizar cada uma em seu contexto específico.
- ❑ Refinar e iterar: O conjunto de componentes é refinado continuamente com feedback do desenvolvimento. Conforme a implementação avança, descobrem-se novos detalhes e casos extremos, então o design de componentes deve evoluir iterativamente. Alterações como dividir componentes grandes ou combinar componentes muito pequenos são normais até atingir um equilíbrio adequado. A arquitetura efetiva emerge de repetidos ajustes, não de um desenho fixo inicial.



GRANULARIDADE DE COMPONENTES

- ❑ Componentes Fine-Grained vs Coarse-Grained: Encontrar o tamanho ideal dos componentes é um dos desafios do arquiteto. Se definirmos componentes muito pequenos e específicos, o sistema sofre com comunicações excessivas entre eles para cumprir funcionalidades, adicionando complexidade e latência. Por outro lado, componentes muito grandes concentram muitas responsabilidades internamente, gerando alto acoplamento interno e dificultando testes e implantação isolada. É preciso buscar um meio-termo onde cada componente realiza uma função coesa sem se tornar monolítico.
- ❑ Heurística de Granularidade: Não há fórmula exata; o ajuste fino geralmente vem da iteração. Começa-se com componentes coerentes com os limites de domínio ou função, e então avalia-se: componentes que se comunicam demais talvez deveriam ser um só, enquanto componentes com responsabilidades muito distintas ou cargas muito desiguais talvez devam ser separados. O objetivo é minimizar dependências entre componentes e, simultaneamente, evitar complexidade excessiva dentro de cada componente.



ANTI-PATTERN: ENTITY TRAP

- ❑ Descrição do Problema: O arquiteto identifica componentes apenas espelhando as entidades do banco de dados, criando, por exemplo, componentes "Manager" para cada entidade (Pedido, Item, Leilão, etc.). O resultado é uma “arquitetura” que na verdade não adiciona abstração significativa além do que um ORM já forneceria (basicamente uma interface direta para CRUD de tabelas).
- ❑ Por que é uma Armadilha?: Essa abordagem ignora os fluxos de trabalho reais do sistema. Os relacionamentos do banco raramente correspondem 1:1 aos processos de negócio. Assim, componentes definidos apenas pelas entidades tendem a ser excessivamente genéricos e granulares demais, não ajudando a equipe a estruturar o código de forma orientada ao domínio. Em suma, falta uma separação clara de responsabilidades orientada às ações do sistema, prejudicando a modularidade e a compreensão da arquitetura.



TÉCNICAS DE DESCOBERTA DE COMPONENTES

- ❑ Abordagem Atores/Ações (Actor/Actions): Técnica derivada do RUP que consiste em identificar os diferentes atores (papéis de usuários ou sistemas externos) e as ações que eles executam no sistema. Cada combinação ator-ação significativa pode sugerir um componente ou funcionalidade principal. Essa abordagem funciona bem quando há distintos papéis de usuário e ajuda a garantir que os componentes reflitam as necessidades de cada tipo de usuário.
- ❑ Event Storming: Técnica popular associada a DDD e microservices. Envolve mapear eventos de negócio que ocorrem no domínio e, a partir deles, derivar componentes responsáveis por reagir a esses eventos. Os participantes (devs, analistas, SMEs) fazem um brainstorming de eventos e comandos numa linha do tempo do processo, descobrindo componentes naturalmente ao agrupar comportamentos em torno de cada evento chave. Útil para arquiteturas orientadas a eventos/mensageria, pois já define possíveis mensagens entre componentes.



TÉCNICAS DE DESCOBERTA DE COMPONENTES

- ❑ Abordagem por Workflows: Similar ao Event Storming, mas mais genérica e não necessariamente focada em eventos explícitos. Consiste em identificar os principais fluxos de trabalho ou casos de uso do sistema e estruturar componentes em torno dessas atividades. É adequada quando não se está adotando DDD ou quando o sistema não é fortemente dirigido a eventos; foca-se em decompor o sistema pelas sequências lógicas de passos (workflows) que entregam valor.
- ❑ Como escolher: Não há uma técnica universalmente melhor. Cada método traz vantagens e possíveis limitações. Projetos mais tradicionais ou com muita clareza de atores tendem a se beneficiar de Atores/Ações, enquanto domínios complexos modelados em DDD favorecem Event Storming, por exemplo. O importante é usar uma técnica que se alinhe ao processo de desenvolvimento adotado e estar ciente de que todas visam gerar um primeiro desenho de componentes, sujeito a refinamento iterativo conforme descobertas forem feitas.



DECISÃO ARQUITETURAL: MONÓLITO VS DISTRIBUÍDO

- ❑ **Arquitetura Monolítica:** Uma aplicação monolítica típica é implantada como uma única unidade (um único processo contendo todos os componentes) geralmente conectada a um banco de dados unificado. Oferece simplicidade de implantação e comunicação interna direta entre componentes, sendo vantajosa se o sistema puder ser satisfeito com um único conjunto de características arquiteturais uniforme. Nesse caso, os benefícios incluem menor complexidade de orquestração, transações fáceis e desempenho sem latência de rede.
- ❑ **Arquitetura Distribuída (Microservices):** Consiste em múltiplos serviços/componentes implantados separadamente, cada um rodando em seu próprio processo e comunicando-se via rede (REST, mensageria, etc.). Adequada quando diferentes componentes do sistema possuem demandas arquiteturais bem distintas. Por exemplo, um componente exigindo alta escalabilidade pode ser isolado de outro que requer alta consistência, permitindo otimizações e escala independentes. Em arquiteturas distribuídas, cada serviço pode ter seu próprio ciclo de deploy e tecnologia, ao custo de uma complexidade maior de integração e monitoração.



DECISÃO ARQUITETURAL: MONÓLITO VS DISTRIBUÍDO

- ❑ Critério de Escolha: A decisão entre permanecer em um monólito ou evoluir para microserviços deve ser guiada pelas características arquiteturais identificadas no design. Se todos os componentes puderem compartilhar requisitos não-funcionais similares, um monólito simplifica o sistema. Mas se surgirem componentes com necessidades incompatíveis (em termos de escalabilidade, disponibilidade, etc.), é um indicativo de partir para uma arquitetura distribuída para isolá-los. Em suma, avaliar os trade-offs é crucial: cada estilo tem prós e contras, e o arquiteto deve optar pela solução com o conjunto de desvantagens menos prejudicial aos objetivos do projeto.



CONCLUSÃO

- ❑ Componentização: Pensar “component-based” significa estruturar o software em blocos físicos modulares que facilitam a manutenção e evolução. Componentes bem definidos aumentam a compreensão do sistema e servem de base para escalabilidade e reutilização.
- ❑ Projeto Evolutivo: A definição de componentes não é etapa única: envolve iteração constante. Arquitetos e times devem refinar os limites dos componentes conforme aprendem mais sobre o domínio e identificam diferentes requisitos de qualidade.
- ❑ Trade-offs Arquiteturais: Não existe partição perfeita. Cada escolha (camadas técnicas vs domínios, monólito vs microserviços) traz vantagens e compromissos. O papel do arquiteto é avaliar objetivamente essas trocas e guiar o projeto rumo à opção cujo conjunto de trade-offs seja o “menos pior” para aquele contexto. Em última instância, uma boa arquitetura é aquela que equilibra as necessidades do negócio com as restrições técnicas, mantendo o sistema o mais simples possível, mas preparado para mudanças futuras.



ATÉ A PRÓXIMA AULA